# UNIT-2

Introduction to SQL (Structured Query Language)

- SQL (Structured Query Language) is a standard language used to interact with relational databases. It allows users to store, retrieve, manipulate, and manage data efficiently.

# History of SQL

- Dr. E.F. Codd published a paper on **Relational Model** ("A Relational Model Data from Large Shared Data Banks") in 1970 in ACM journal. Using the Mathematical concepts of Relational Algebra and Tuple Relational Calculus. This model was accepted as definitive model for **RDBMS**.
- **IBM** implemented the **SQL language**, originally called **SEQUEL** (**Structured English Query Languag** as part of the ***System R*** project in the early **1970s**.
  - **SEQUEL** is renamed/shortened to **SQL (Structured Query Language)**
- **SQL** became a standard of the ***American National Standards Institute (ANSI)*** in **1986**, and of the ***International Organization for Standardization (ISO)*** in **1987**.
- **ANSI** and **ISO** standard SQL has different **versions** :
  - SQL-86, SQL-89, SQL-92 , SQL:1999 , SQL:2003, SQL:2006, SQL:2008, SQL:2011, SQL:2016
- Vendors of Commercial systems (like Oracle, SQL Server etc.) offers most of the **major features**, plu **varying feature sets** from later standards and special proprietary **extensions.**

# SQL (Structured Query Language)

- **SQL** is a **Domain-Specific Language.**
- **SQL** is **Declarative language i.e.** a **non-procedural language.**
  - **SQL** allows to declare **what we want to do**, but not how to do.
    - In **SQL**, we can specify through queries that **what data we want** but does not specify how to get those data.
    - Whereas, **C** is procedural language as it uses functions, procedures, loops, etc. **to specify what to do and how to do it.**

**Question:** How the query is actually performed in SQL?
**Answer:** The database/SQL engine is very powerful. When we write an SQL query, it first parses it; then it figures out its internal algorithms and it tries to select an algorithm which will be the best for that particular query. So, then it applies that algorithm, finds the answer and returns it.

- **SQL** is based on ***Relational Algebra*** and ***Tuple Relational Calculus.***

# SQL Rules

- **SQL** is **NOT case sensitive.**
  - For example: select is the same as SELECT
- But *names* of **databases, tables and columns** are **case sensitive**
  - For example: In given SQL query, ACCOUNT is different from account

```
SELECT * FROM ACCOUNT;
SELECT * FROM account;
```

- **SQL statements** can use **multiple lines**
  - End each SQL statement with a **semi-colon** ;

# Why SQL?

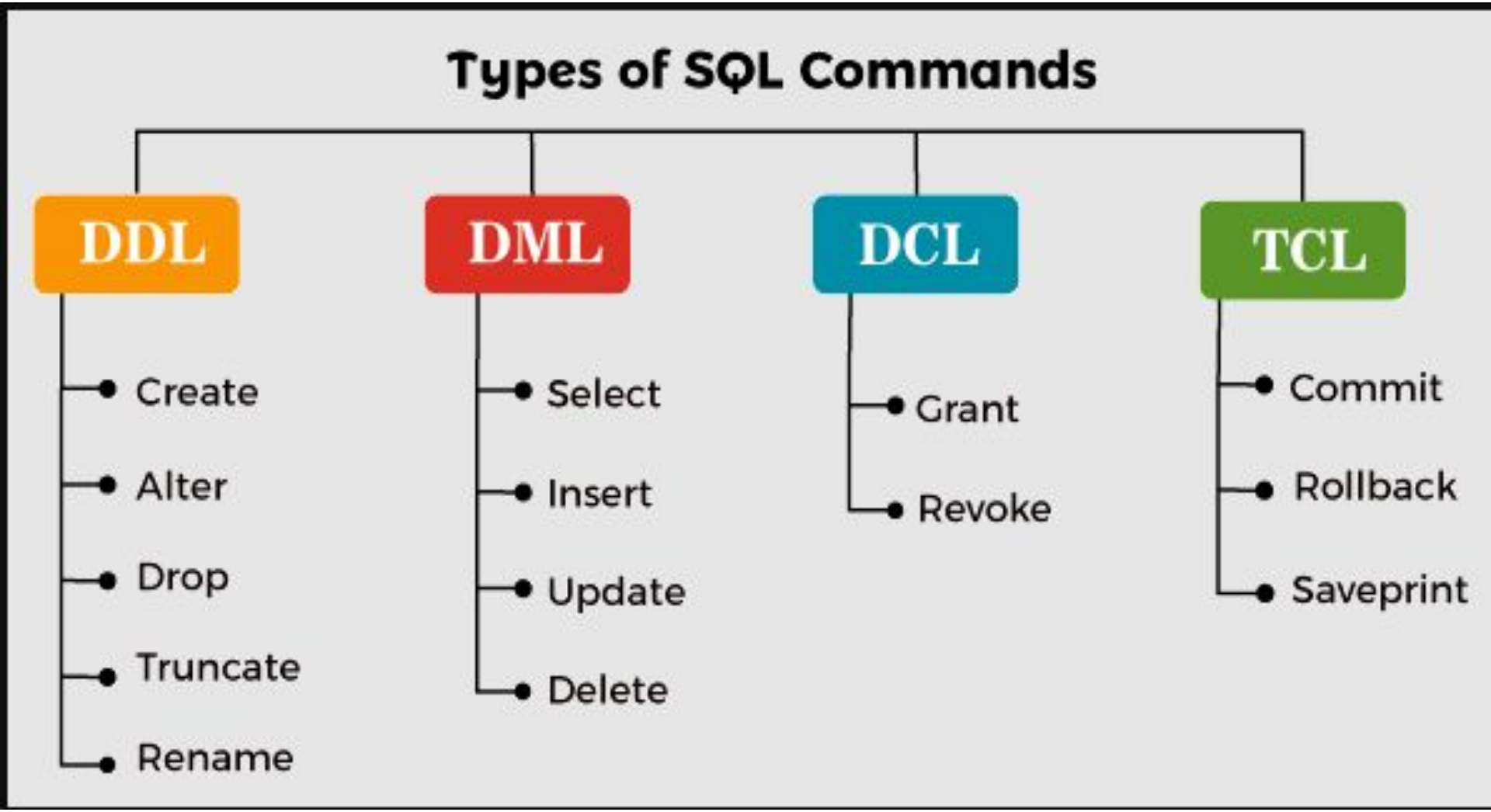
- SQL is widely used for:
  - **Data Retrieval** – Fetching data from databases.
  - **Data Manipulation** – Inserting, updating, and deleting records.
  - **Database Management** – Creating and modifying database structures.
  - **Security & Access Control** – Managing user permissions.

Popular database management systems (DBMS) that use SQL:

- **MySQL** – Open-source and widely used.
- **PostgreSQL** – Advanced open-source database.
- **Microsoft SQL Server** – Enterprise-level solution.
- **Oracle Database** – Used in large-scale applications.
- **SQLite** – Lightweight and embedded database.

# SQL Commands

- SQL commands are instructions
- It is used to communicate with the database.
- It is also used to perform specific tasks,Functions & queries of data.
- SQL can perform various tasks like create, add, drop, modify the table & set permission for users.

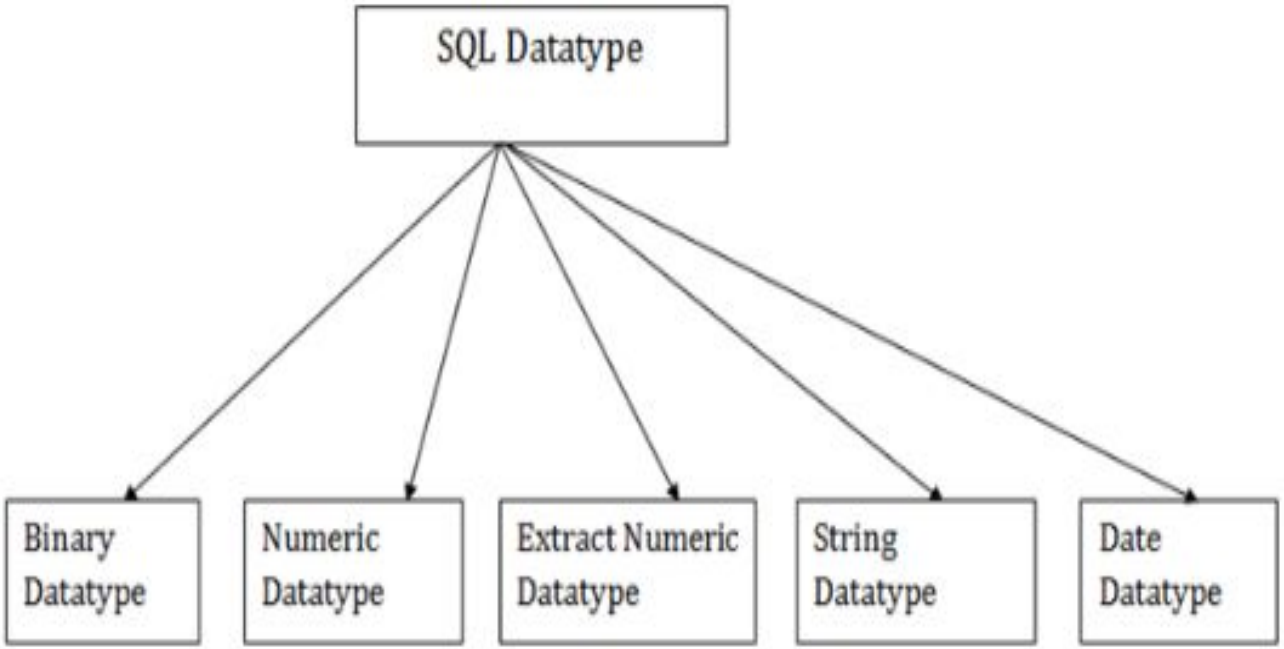

Types of SQL Commands
DDL
Create
Alter
Drop
Truncate
Rename
DML
Select
Insert
Update
Delete
DCL
Grant
Revoke
TCL
Commit
Rollback
Saveprint

# SQL commands are categorized into different groups:

1. **Data Definition Language (DDL)**
   - CREATE – Creates a new table or database.
   - ALTER – Modifies an existing table.
   - DROP – Deletes a table or database.
2. **Data Manipulation Language (DML)**
   - INSERT – Adds new records
   - UPDATE – Modifies existing records.
   - DELETE – Removes records.
3. **Data Control Language (DCL)**
   - GRANT – Provides access privileges.
   - REVOKE – Removes access privileges.
4. **Transaction Control Language (TCL)**
   - COMMIT – Saves all changes.
   - ROLLBACK – Undoes changes.
   - SAVEPOINT – Creates a rollback point.

# SQL Datatype

- SQL Datatype is used to define the values that a column can contain.
- Every column is required to have a name and



- 1. Binary Datatypes
  - There are Three types of binary Datatypes which are given below:

| Data Type | Description |
|---|---|
| binary | It has a maximum length of 8000 bytes. It contains fixed-length binary data. |
| varbinary | It has a maximum length of 8000 bytes. It contains variable-length binary data. |
| image | It has a maximum length of 2,147,483,647 bytes. It contains variable-length binary data. |

## 2. Approximate Numeric Datatype :

The subtypes are given below:

| Data type | Description |
|---|---|
| float | It is used to specify a floating-point value e.g. 6.2, 2.9 etc. |
| real | It specifies a single precision floating point number |

# 3. Exact Numeric Datatype

The subtypes are given below:

| Data type | Description |
|---|---|
| int | It is used to specify an integer value. |
| smallint | It is used to specify small integer value. |
| bit | It has the number of bits to store. |
| decimal | It specifies a numeric value that can have a decimal number. |
| numeric | It is used to specify a numeric value. |

# 4. Character String Data type

The subtypes are given below:

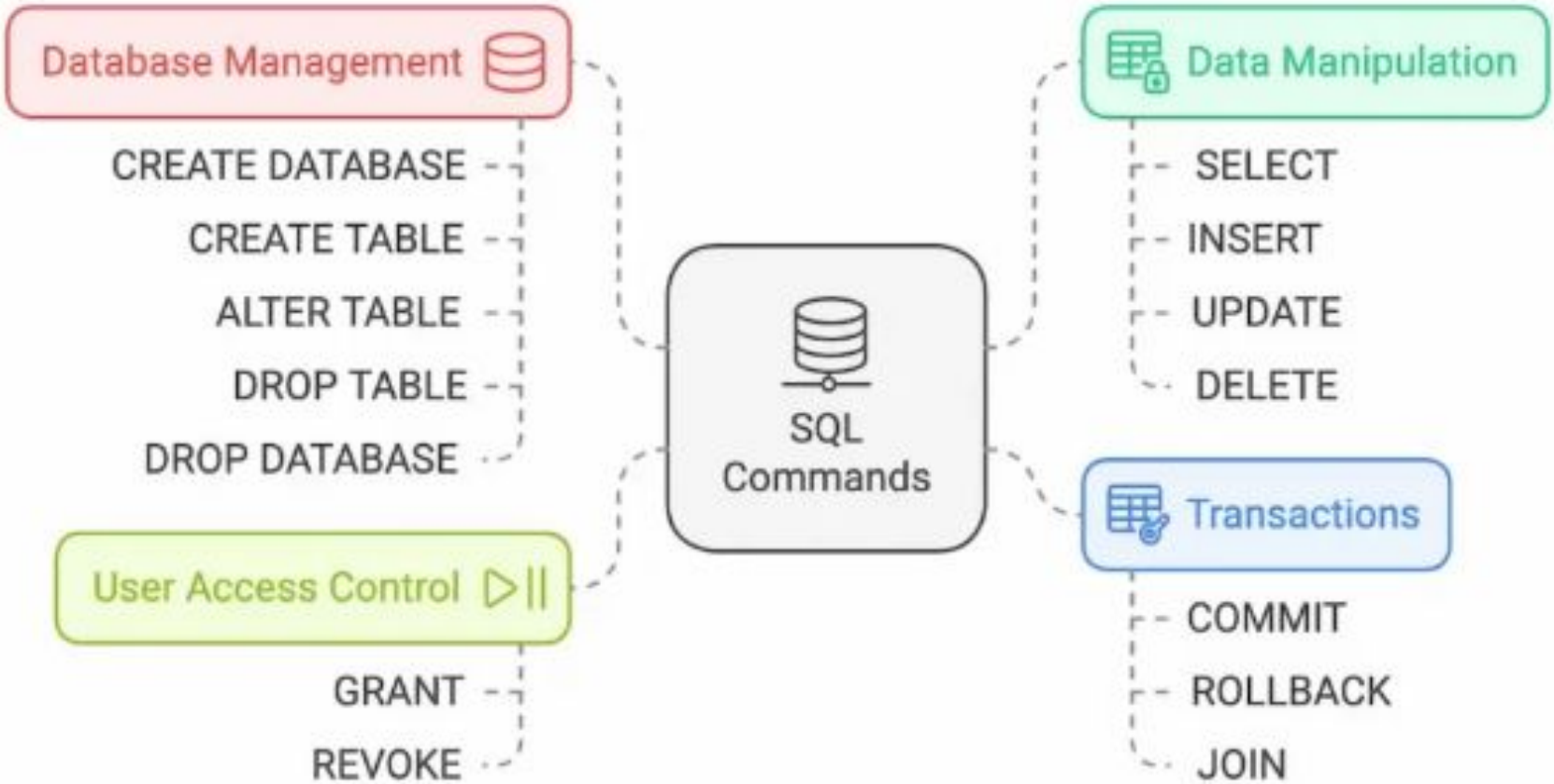
| Data type | Description |
|---|---|
| char | It has a maximum length of 8000 characters. It contains Fixed-length non-unicode characters. |
| varchar | It has a maximum length of 8000 characters. It contains variable-length non-unicode characters. |
| text | It has a maximum length of 2,147,483,647 characters. It contains variable-length non-unicode characters. |

# 5. Date and time Datatypes

The subtypes are given below:

| Datatype | Description |
| --- | --- |
| date | It is used to store the year, month, and days value. |
| time | It is used to store the hour, minute, and second values. |
| timestamp | It stores the year, month, day, hour, minute, and the second value. |

# SQL Commands and Their Functions



# Data Definition Language (DDL)

- DDL changes the structure of the table like creating a table, deleting a table, altering a table, etc.
- All the command of DDL are auto-committed that means it permanently save all the changes in the database.

- **CREATE** It is used to create a new table in the database.

```
CREATE TABLE TABLE_NAME (COLUMN_NAMES DATATYPES [ ...]);
```

```
CREATE TABLE EMPLOYEE(Name VARCHAR2(20), Email VARCHA
R2(100), DOB DATE);
```

**DROP:** It is used to delete both the structure and record stored in the table.

```
DROP TABLE EMPLOYEE;
```

**Syntax: To DROP a table permanently from memory**

**ALTER:** It is used to alter the structure of the database. This change could be either to modify the characteristics of an existing attribute or probably to add a new attribute.

- **Following are the list of modifications that can be done using ALTER command.**
  - With the use of ALTER commands we can add or drop one or more columns form existing tables.
  - Increase or decrease the existing column width by changing the data type
  - Make an existing mandatory column to optional.
  - Enable or disable the integrity constraints in a table. We can also add, modify or delete the integrity constraints from a table.
  - We can also specify a default value for existing column in a table.

- **TRUNCATE:** It is used to delete all the rows from the table and free the space containing the table.

  **Rename:** It is used to rename the table.

# Data Manipulation Language

- DML is used for inserting, deleting, and updating data in a database. It is used to retrieve and manipulate data in a relational database.
- It includes INSERT, UPDATE, and DELETE.

**a. INSERT:** The INSERT statement is a SQL query. It is used to insert data into the row of a table.

**Syntax: To add row in a table**

```
INSERT INTO TABLE_NAME
(col1, col2, col3,.... col N)
VALUES (value1, value2, value3, .... valueN);
```

Or

```
INSERT INTO TABLE_NAME
VALUES (value1, value2, value3, .... valueN);
```

**b. UPDATE:** This command is used to update or modify the value of a column in the table.

**Syntax: To update record in a table**

```
UPDATE table_name SET [column_name1= value1,...column_nam
eN = valueN] [WHERE CONDITION]
```

**For example:**

```
UPDATE students
SET User_Name = 'Sonoo'
WHERE Student_Id = '3'
```

## c. **DELETE:** It is used to remove one or more row from a table.

**Syntax: To Delete a record from table**

```
DELETE FROM table_name [WHERE condition];
```

**For example:**

```
DELETE FROM Student
WHERE Author="Sonoo";
```

**d. SELECT:** This is the same as the projection operation of relational algebra. It is used to select the attribute based on the condition described by WHERE clause.

**Syntax: It is used for retrieve the records from table**

```
SELECT expressions
FROM TABLES
WHERE conditions;
```

**For example:**

```
SELECT emp_name
FROM employee
WHERE age > 20;
```

# 3. Data Control Language

- DCL commands are used to grant and take back authority from any database user.

**Syntax:**

```
GRANT <obj_priv> ON <obj_name> To <username>;
```

GRANT SELECT, UPDATE ON MY_TABLE TO SOME_USER, ANOTHER_USER;

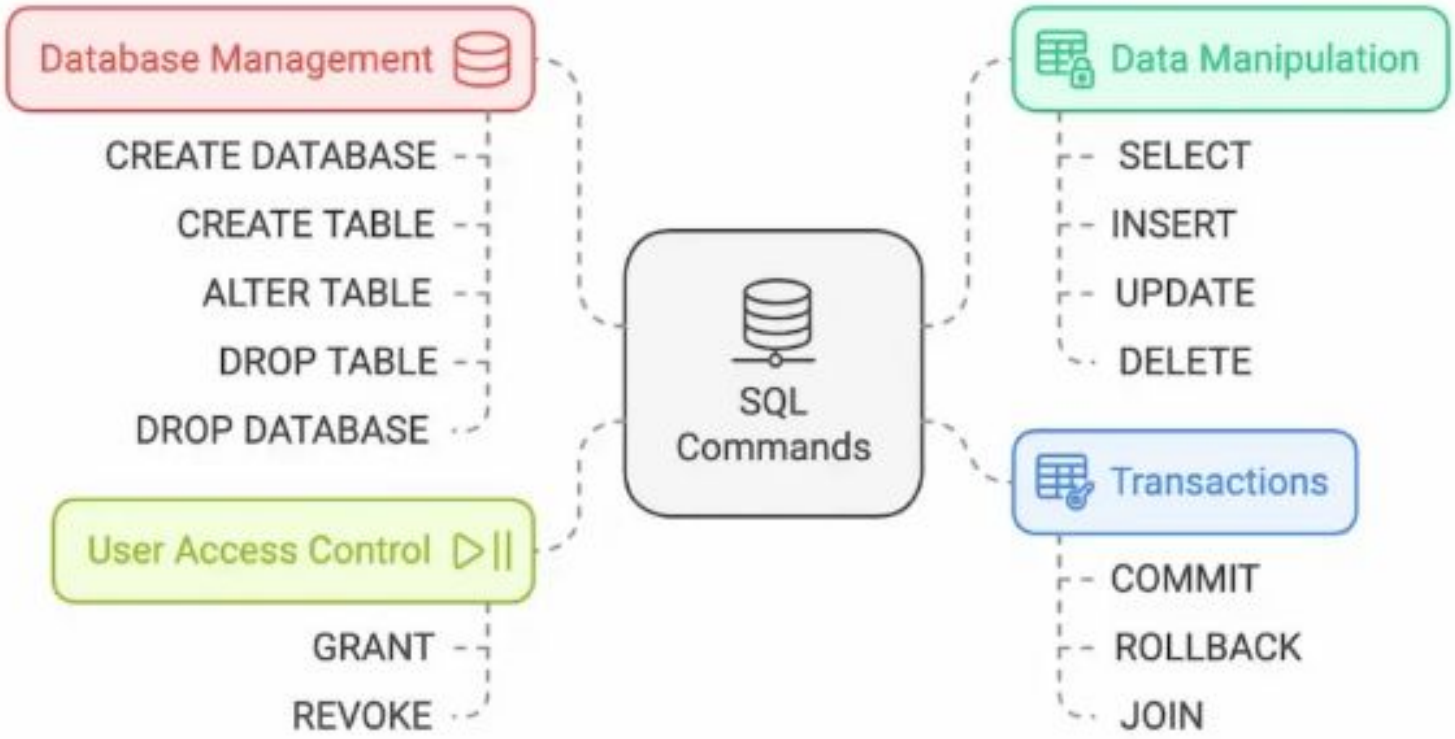
**b. Revoke:** It is used to take back permissions from the user.
**Syntax:**

```
REVOKE <obj_priv> ON <obj_name> FROM <username>;
```

**Example**

```
REVOKE SELECT, UPDATE ON MY_TABLE FROM USER1, USER2;
```

# SQL Commands and Their Functions



- **COMMIT:** Commits the current transaction, making all changes made during the transaction permanent.
- **ROLLBACK:** Reverts changes back to the last commit point. It’s used to undo transactions that have not yet been committed.
- **JOIN:** Combines rows from two or more tables based on a related column between them.

# What is logical operators in SQL?

- *Logical operators in SQL are used to combine multiple conditions in a query to evaluate their truthfulness, returning TRUE, FALSE, or UNKNOWN.*

*The three basic logical operators are AND, OR, and NOT, which help combine or negate conditions in SQL queries.*

- SQL **Logical Operators** are essential tools used to test the truth of conditions in **SQL queries**. They return **boolean values** such as **TRUE**, **FALSE**, or **UNKNOWN**, making them invaluable for **filtering**, **retrieving**, or **manipulating** data.
- These operators allow developers to build **complex queries** by combining, negating, or comparing conditions effectively.

# • 1. AND Operator

- The **AND operator** is used to combine **two** or **more conditions** in an SQL query. It returns records only when **all conditions** specified in the query are **true**. This operator is commonly used when **filtering data** that must satisfy multiple criteria simultaneously.
- **Example**
- Retrieve the records of employees from the employees table who are located in '**Allahabad**' and belong to '**India**', ensuring that both conditions are met.
- **Query:**

```
SELECT * FROM employee WHERE emp_city = 'Allahabad'
AND emp_country = 'India';
```

| emp_id | emp_name | emp_city | emp_country |
|---|---|---|---|
| 101 | Utkarsh Tripathi | Varanasi | India |
| 102 | Abhinav Singh | Varanasi | India |
| 103 | Utkarsh Raghuvanshi | Varanasi | India |
| 104 | Utkarsh Singh | Allahabad | India |
| 105 | Sudhanshu Yadav | Allahabad | India |
| 106 | Ashutosh Kumar | Patna | India |

```
SELECT * FROM employee WHERE emp_city = 'Allahabad'
AND emp_country = 'India';
```

| emp_id | emp_name | emp_city | emp_country |
|---|---|---|---|
| 104 | Utkarsh Singh | Allahabad | India |
| 105 | Sudhanshu Yadav | Allahabad | India |

# 2. OR Operator

- The **OR operator** combines multiple conditions in a SQL query and returns **TRUE** if at least one of the conditions is satisfied. It is ideal for situations where you want to retrieve records that meet any of several possible conditions.

**Example**

Retrieve the records of employees from the `employee` table who are either from `'Varanasi'` or have `'India'` as their country.

## Query

```
SELECT * FROM employee WHERE emp_city = 'Varanasi'
OR emp_country = 'India';
```

| emp_id | emp_name | emp_city | emp_country |
|---|---|---|---|
| 101 | Utkarsh Tripathi | Varanasi | India |
| 102 | Abhinav Singh | Varanasi | India |
| 103 | Utkarsh Raghuvanshi | Varanasi | India |
| 104 | Utkarsh Singh | Allahabad | India |
| 105 | Sudhanshu Yadav | Allahabad | India |
| 106 | Ashutosh Kumar | Patna | India |

# 3. NOT Operator

- The **NOT operator** is used to reverse the result of a condition, returning **TRUE** when the condition is **FALSE**. It is typically used to exclude records that match a specific condition, making it useful for filtering out unwanted data.

**Example:-**

Retrieve the records of employees from the employee table whose city names do not start with the letter 'A'.

**Query:**

```
SELECT * FROM employee WHERE emp_city NOT LIKE
'A%';
```

| emp_id | emp_name | emp_city | emp_country |
|---|---|---|---|
| 101 | Utkarsh Tripathi | Varanasi | India |
| 102 | Abhinav Singh | Varanasi | India |
| 103 | Utkarsh Raghuvanshi | Varanasi | India |
| 106 | Ashutosh Kumar | Patna | India |

# 4. IN Operator

- The IN operator simplifies the process of checking if a **value matches any value in a list**, making it more efficient and readable compared to using multiple **OR** conditions.
- This operator is especially helpful when we need to filter results based on **multiple possible values** for a given column, reducing the complexity of the query.

**Example:-** Retrieve the records of employees from the employee table who are located in either '**Allahabad**' or '**Patna**'.

| emp_id | emp_name | emp_city | emp_country |
|---|---|---|---|
| 101 | Utkarsh Tripathi | Varanasi | India |
| 102 | Abhinav Singh | Varanasi | India |
| 103 | Utkarsh Raghuvanshi | Varanasi | India |
| 104 | Utkarsh Singh | Allahabad | India |
| 105 | Sudhanshu Yadav | Allahabad | India |
| 106 | Ashutosh Kumar | Patna | India |

**Query:**

```
SELECT * FROM employee WHERE emp_city IN
('Allahabad', 'Patna');
```

**Query:**

```
SELECT * FROM employee WHERE emp_city IN
('Allahabad', 'Patna');
```

| emp_id | emp_name | emp_city | emp_country |
|---|---|---|---|
| 104 | Utkarsh Singh | Allahabad | India |
| 105 | Sudhanshu Yadav | Allahabad | India |
| 106 | Ashutosh Kumar | Patna | India |

# 5. BETWEEN Operator

- The **BETWEEN operator** in SQL allows us to test if a value or expression lies within a **specified range**. The **BETWEEN** condition is inclusive, meaning it includes both the **lower** and in the results.
- This operator is particularly useful when we need to filter records based on a range of values, such as numerical ranges, dates, or even text values.

**Example:-** Retrieve the records of employees from the employee table whose emp_id values fall within the range of 101 to 104 (inclusive).

**Query:**

```
SELECT * FROM employee WHERE emp_id BETWEEN 101 AND
104;
```

| emp_id | emp_name | emp_city | emp_country |
|---|---|---|---|
| 101 | Utkarsh Tripathi | Varanasi | India |
| 102 | Abhinav Singh | Varanasi | India |
| 103 | Utkarsh Raghuvanshi | Varanasi | India |
| 104 | Utkarsh Singh | Allahabad | India |

**Example:-** Retrieve the records of employees from the employee table whose emp_id values fall within the range of 101 to 104 (exclude).

```
SELECT * FROM employee WHERE emp_id > 101
  AND emp_id < 104;
```

# NULL values in SQL

- Disallowing Null Values
- Comparison Using Null Values

In SQL, handling NULL values properly is crucial, especially when filtering records.

Here’s how you can modify your query to **disallow NULL values in emp_id** and avoid unintended behavior when comparing with NULL:

# Disallowing null values

- Disallowing null values means ensuring that a field or column in a database or data structure cannot contain NULL (i.e., missing or undefined values). This is typically enforced by defining constraints such as NOT NULL in SQL databases, using strict data validation rules in programming languages, or configuring schemas in formats like JSON or XML.

# Example 1: Creating a Table with NOT NULL

```
CREATE TABLE Employees (
    EmployeeID INT PRIMARY KEY,
    Name VARCHAR(100) NOT NULL,
    Email VARCHAR(255) NOT NULL,
    Age INT
);
```

# Comparison Using NULL Values in SQL

- In SQL, NULL represents an **unknown or missing value**. Because of this, comparisons involving NULL behave differently than regular values.

## 1. NULL in Comparisons (=, !=, <, >)

Any comparison with NULL results in**UNKNOWN** (NULL).

Example:

```
SELECT * FROM Employees WHERE Age = NULL;
```

This **does not** return any rows because **NULL** cannot be compared using =

Query:

```sql
SELECT *
FROM employee
WHERE emp_id > 101 AND emp_id < 104
AND emp_id IS NOT NULL;
```

## Explanation:

1. **emp_id > 101 AND emp_id < 104**: Ensures that only employees with emp_id between 102 and 103 are retrieved.

2. **emp_id IS NOT NULL**: Explicitly filters out any records where emp_id is NULL, since NULL values can cause unexpected behavior in comparisons.

- **2. Using IS NULL and IS NOT NULL**

To properly check for NULL, use IS NULL or IS NOT NULL:

```
SELECT * FROM Employees WHERE Age IS NULL;
```

- Retrieves rows where Age is NULL.

```
SELECT * FROM Employees WHERE Age IS NOT NULL;
```

# Integrity Constraints in SQL

- **Integrity constraints** are rules enforced on a database to ensure the **accuracy, consistency, and validity** of the data. These constraints help maintain data integrity by preventing invalid or inconsistent data from being entered into the database.

Examples of integrity constraints:

- **Entity integrity**: Ensures that the primary key cannot be null
- **Check constraints**: Enforces rules on the values in a column
- **Unique constraints**: Ensures that all values in a column are distinct
- **Referential integrity**: Ensures that relationships between tables remain consistent
- **Domain constraints**: Constrains the value of a single column